

# Personalized Recommendation Layer

## Technical Architecture & Implementation Guide

Engineering Team

February 2026

### Contents

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>System Overview</b>                                               | <b>3</b>  |
| 1.1      | Architecture Principles . . . . .                                    | 3         |
| 1.2      | High-Level Architecture . . . . .                                    | 3         |
| <b>2</b> | <b>Data Model</b>                                                    | <b>3</b>  |
| 2.1      | Feedback Schema . . . . .                                            | 3         |
| 2.2      | Personalization Features . . . . .                                   | 4         |
| <b>3</b> | <b>Personalization Models</b>                                        | <b>4</b>  |
| 3.1      | Model 1: GLMix (Generalized Linear Mixed Model) . . . . .            | 4         |
| 3.1.1    | Mathematical Formulation . . . . .                                   | 4         |
| 3.1.2    | Feature Engineering . . . . .                                        | 5         |
| 3.1.3    | Training Algorithm . . . . .                                         | 6         |
| 3.2      | Model 2: Multi-Armed Bandit (for Cold Start & Exploration) . . . . . | 6         |
| 3.2.1    | Thompson Sampling Algorithm . . . . .                                | 6         |
| 3.2.2    | Exploration-Exploitation Tradeoff . . . . .                          | 6         |
| 3.3      | Model 3: Contextual Bandit (LinUCB) . . . . .                        | 7         |
| <b>4</b> | <b>Implementation Details</b>                                        | <b>7</b>  |
| 4.1      | Offline Training Pipeline . . . . .                                  | 7         |
| 4.2      | Online Serving Pipeline . . . . .                                    | 9         |
| 4.3      | Feedback Collection . . . . .                                        | 12        |
| <b>5</b> | <b>Explainability Module</b>                                         | <b>13</b> |
| 5.1      | Feature Attribution with SHAP . . . . .                              | 13        |
| <b>6</b> | <b>Bias Mitigation &amp; Fairness</b>                                | <b>14</b> |
| 6.1      | Protected Attribute Filtering . . . . .                              | 14        |
| 6.2      | Disparate Impact Monitoring . . . . .                                | 14        |
| 6.3      | Diversity Injection . . . . .                                        | 15        |
| <b>7</b> | <b>Performance Optimization</b>                                      | <b>16</b> |
| 7.1      | Caching Strategy . . . . .                                           | 16        |
| 7.2      | Async Processing . . . . .                                           | 16        |
| <b>8</b> | <b>Monitoring &amp; Observability</b>                                | <b>17</b> |
| 8.1      | Key Metrics to Track . . . . .                                       | 17        |
| 8.2      | Alerting Rules . . . . .                                             | 18        |

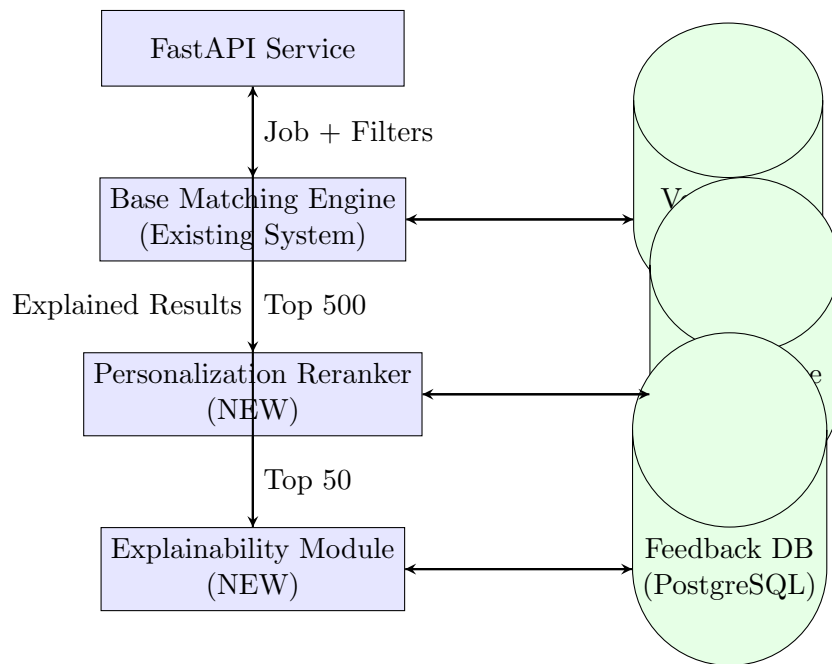
|           |                                       |           |
|-----------|---------------------------------------|-----------|
| <b>9</b>  | <b>Deployment Strategy</b>            | <b>18</b> |
| 9.1       | Rollout Plan . . . . .                | 18        |
| 9.2       | A/B Testing Framework . . . . .       | 18        |
| <b>10</b> | <b>Technology Stack</b>               | <b>19</b> |
| 10.1      | Core Components . . . . .             | 19        |
| 10.2      | Infrastructure Requirements . . . . . | 19        |
| <b>11</b> | <b>Testing Strategy</b>               | <b>20</b> |
| 11.1      | Unit Tests . . . . .                  | 20        |
| 11.2      | Integration Tests . . . . .           | 20        |
| 11.3      | Load Testing . . . . .                | 21        |
| <b>12</b> | <b>Conclusion</b>                     | <b>21</b> |

# 1 System Overview

## 1.1 Architecture Principles

1. **Non-Disruptive:** Recommendation layer sits on top of existing matching system
2. **Graceful Degradation:** Falls back to base model when personalization unavailable
3. **Low Latency:**  $\leq 50$ ms additional overhead for recommendation layer
4. **Explainable:** Every recommendation includes feature attribution
5. **Fair:** Built-in bias monitoring and diversity constraints

## 1.2 High-Level Architecture



# 2 Data Model

## 2.1 Feedback Schema

```

CREATE TABLE feedback_events (
  event_id UUID PRIMARY KEY,
  company_id UUID NOT NULL,
  recruiter_id UUID NOT NULL,
  job_id UUID NOT NULL,
  candidate_id UUID NOT NULL,
  event_type VARCHAR(50) NOT NULL, -- 'view', 'shortlist', 'reject', '
    interview', 'hire'
  event_timestamp TIMESTAMP NOT NULL,
  implicit_score FLOAT, -- 0.0 to 1.0, computed from behavior
  explicit_rating INT, -- Optional: 1-5 star rating
  session_id UUID, -- For in-session learning
  metadata JSONB, -- Flexible additional context
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_company_job ON feedback_events(company_id, job_id);

```

```
CREATE INDEX idx_candidate ON feedback_events(candidate_id);
CREATE INDEX idx_event_type ON feedback_events(event_type);
CREATE INDEX idx_timestamp ON feedback_events(event_timestamp);
```

## 2.2 Personalization Features

```
class PersonalizationFeatures:
    """Features extracted from company hiring history"""

    # Location preferences
    location_preferences: Dict[str, float] # {'Bangalore': 0.8, 'Delhi':
        0.6, ...}

    # Skill preferences (beyond JD requirements)
    skill_affinities: Dict[str, float] # {'Docker': 0.7, 'AWS': 0.9, ...}

    # Experience patterns
    experience_range_preference: Tuple[float, float] # (min, max) years
    job_switch_preference: float # Prefer stable (-1) or dynamic (+1)
        careers

    # Education patterns
    college_tier_preference: Dict[str, float] # {'tier1': 0.9, 'tier2':
        0.4, ...}
    degree_preference: Dict[str, float] # {'BTech': 0.8, 'MTech': 0.6,
        ...}

    # Industry preferences
    industry_affinities: Dict[str, float] # {'fintech': 0.9, 'edtech':
        0.3, ...}

    # Temporal patterns
    recency_weight: float # How much to weight recent vs. old experience
    hiring_velocity: float # Fast hires (days) indicates urgency

    # Two-sided signals
    candidate_response_rate: float # Likelihood candidate will respond
    offer_acceptance_rate: float # Likelihood candidate will accept offer
```

## 3 Personalization Models

### 3.1 Model 1: GLMix (Generalized Linear Mixed Model)

#### 3.1.1 Mathematical Formulation

**Goal:** Learn a personalized scoring function per company/recruiter

$$\text{score}(c, j, r) = \underbrace{\beta_{\text{global}}^T x_{cj}}_{\text{Global Model}} + \underbrace{\beta_r^T x_{cj}}_{\text{Recruiter Effect}} + \underbrace{\beta_{r,c}^T x_{cj}}_{\text{Company Effect}} + \epsilon \quad (1)$$

where: (2)

$c$  = candidate (3)

$j$  = job (4)

$r$  = recruiter/company (5)

$x_{cj}$  = feature vector (from base matching model) (6)

$\beta_{\text{global}}$  = global coefficients (learned from all data) (7)

$\beta_r$  = recruiter-specific coefficients (8)

$\beta_{r,c}$  = company-specific coefficients (9)

$\epsilon \sim \mathcal{N}(0, \sigma^2)$  (10)

### 3.1.2 Feature Engineering

```
def extract_glmix_features(candidate, job, company_history):
    """Extract features for GLMix model"""

    features = {
        # From base matching model
        'skill_match_score': cosine_similarity(candidate.skills_vector,
                                                job.skills_vector),
        'experience_match_score': cosine_similarity(candidate.exp_vector,
                                                    job.role_vector),
        'cv_match_score': cosine_similarity(candidate.cv_vector, job.
                                            jd_vector),

        # Location features
        'location_exact_match': int(candidate.location == job.location),
        'location_city_match': int(candidate.city == job.city),
        'location_remote_compatible': int(candidate.remote_exp > 0 and job
                                           .remote_ok),

        # Experience features
        'experience_years': candidate.total_experience,
        'experience_gap': candidate.total_experience - job.min_experience,
        'num_job_switches': len(candidate.experience) - 1,
        'avg_tenure': candidate.total_experience / max(1, len(candidate.
                                                         experience)),
        'recency_score': np.exp(-0.1 * months_since_last_role(candidate)),

        # Education features
        'college_tier': get_college_tier(candidate.education[0].college),
        'degree_level': encode_degree(candidate.education[0].degree),
        'graduation_year': candidate.education[0].year,

        # Skill diversity
        'num_skills': len(candidate.skills),
        'skill_diversity': entropy(candidate.skill_categories),

        # Industry alignment
        'industry_match': int(candidate.current_industry == job.domain),
        'industry_overlap': jaccard(candidate.industries, [job.domain]),
```

```

    # Interaction features (from GBDT tree features)
    'skill_exp_interaction': skill_match * experience_match,
    'location_skill_interaction': location_match * skill_match,
}

# Company-specific historical features
if company_history:
    features.update({
        'location_hist_match': company_history.location_affinity(
            candidate.location),
        'college_hist_match': company_history.college_affinity(
            candidate.education[0].college),
        'industry_hist_match': company_history.industry_affinity(
            candidate.current_industry),
    })

return features

```

### 3.1.3 Training Algorithm

---

**Algorithm 1** GLMix Training with Block Coordinate Descent

---

```

1: Input: Feedback data  $D = \{(c_i, j_i, r_i, y_i)\}_{i=1}^N$ , where  $y_i \in \{0, 1\}$  (hire/no-hire)
2: Output:  $\beta_{\text{global}}, \{\beta_r\}_r, \{\beta_{r,c}\}_{r,c}$ 
3:
4: Initialize  $\beta_{\text{global}}, \beta_r, \beta_{r,c}$  randomly
5: for epoch = 1 to max_epochs do
6:     // Update global coefficients
7:      $\beta_{\text{global}} \leftarrow \arg \min_{\beta} \sum_i \mathcal{L}(y_i, \beta^T x_i + \beta_{r_i}^T x_i + \beta_{r_i, c_i}^T x_i) + \lambda_1 \|\beta\|_2^2$ 
8:
9:     for each recruiter  $r$  do
10:        // Update recruiter-specific coefficients
11:         $D_r \leftarrow \{(c_i, j_i, y_i) : r_i = r\}$ 
12:         $\beta_r \leftarrow \arg \min_{\beta} \sum_{i \in D_r} \mathcal{L}(y_i, \beta_{\text{global}}^T x_i + \beta^T x_i + \beta_{r, c_i}^T x_i) + \lambda_2 \|\beta\|_2^2$ 
13:    end for
14:
15:    for each company  $c$  do
16:        // Update company-specific coefficients
17:         $D_c \leftarrow \{(c_i, j_i, r_i, y_i) : c_i \in \text{company } c\}$ 
18:         $\beta_{r,c} \leftarrow \arg \min_{\beta} \sum_{i \in D_c} \mathcal{L}(y_i, \beta_{\text{global}}^T x_i + \beta_{r_i}^T x_i + \beta^T x_i) + \lambda_3 \|\beta\|_2^2$ 
19:    end for
20: end for
21: return  $\beta_{\text{global}}, \{\beta_r\}_r, \{\beta_{r,c}\}_{r,c}$ 

```

---

**Loss Function:** Binary cross-entropy (for hire/no-hire)

$$\mathcal{L}(y, \hat{y}) = -y \log(\sigma(\hat{y})) - (1 - y) \log(1 - \sigma(\hat{y})) \quad (11)$$

where  $\sigma(x) = \frac{1}{1+e^{-x}}$  is the sigmoid function.

## 3.2 Model 2: Multi-Armed Bandit (for Cold Start & Exploration)

### 3.2.1 Thompson Sampling Algorithm

### 3.2.2 Exploration-Exploitation Tradeoff

- **Exploitation:** Show candidates from historically successful clusters

**Algorithm 2** Thompson Sampling for Candidate Recommendation

---

```

1: Input: Job  $j$ , candidate pool  $C$ , prior parameters  $\alpha_0, \beta_0$ 
2: Output: Ranked list of candidates
3:
4: // Group candidates into "arms" (e.g., by skill cluster)
5:  $A \leftarrow \text{cluster\_candidates}(C, \text{method} = \text{kmeans}, k = 10)$ 
6:
7: for each arm  $a \in A$  do
8:    $n_a^+ \leftarrow$  number of successful hires from arm  $a$ 
9:    $n_a^- \leftarrow$  number of rejections from arm  $a$ 
10:
11:   // Sample from Beta distribution (Bayesian posterior)
12:    $\theta_a \sim \text{Beta}(\alpha_0 + n_a^+, \beta_0 + n_a^-)$ 
13: end for
14:
15: // Select arm with highest sampled probability
16:  $a^* \leftarrow \arg \max_a \theta_a$ 
17:
18: // Rank candidates within selected arm using GLMix
19:  $\text{candidates} \leftarrow A[a^*]$ 
20:  $\text{scores} \leftarrow [\text{GLMix\_score}(c, j) \text{ for } c \in \text{candidates}]$ 
21: return  $\text{rank\_by\_score}(\text{candidates}, \text{scores})$ 

```

---

- **Exploration:** Occasionally try candidates from uncertain clusters
- **Balance:** Thompson Sampling naturally balances via uncertainty in Beta distribution

**Cold Start Strategy:**

1. New company: Use global priors  $\alpha_0 = 1, \beta_0 = 1$  (uniform prior)
2. After 5 hires: Start personalization with company-specific priors
3. After 20 hires: Full GLMix personalization

**3.3 Model 3: Contextual Bandit (LinUCB)**

For more advanced exploration, use Upper Confidence Bound with linear models:

$$\text{UCB}_a(t) = \theta_a^T x_t + \alpha \sqrt{x_t^T A_a^{-1} x_t} \quad (12)$$

where:

- $\theta_a$ : Learned weights for arm  $a$
- $x_t$ : Context vector (candidate features)
- $A_a$ : Empirical covariance matrix for arm  $a$
- $\alpha$ : Exploration parameter (typically 0.1-0.5)

**4 Implementation Details****4.1 Offline Training Pipeline**

```

# training_pipeline.py

import pandas as pd
from sklearn.preprocessing import StandardScaler
from glmix import GLMixModel
import pickle

def train_personalization_models():
    """Offline training pipeline (runs daily)"""

    # 1. Extract feedback data
    feedback_df = load_feedback_from_db(last_n_days=90)

    # 2. Feature engineering
    features_df = extract_features(feedback_df)
    X = features_df.drop(['event_id', 'label'], axis=1)
    y = features_df['label'] # 1 = hire, 0 = no-hire

    # 3. Train global model
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    global_model = LogisticRegression(penalty='l2', C=1.0)
    global_model.fit(X_scaled, y)

    # 4. Train GLMix models per company
    glmix = GLMixModel(
        global_model=global_model,
        entity_levels=['recruiter_id', 'company_id'],
        regularization={'recruiter_id': 0.1, 'company_id': 0.05}
    )

    glmix.fit(X_scaled, y, entity_ids=feedback_df[['recruiter_id', 'company_id']])

    # 5. Train bandit models
    bandit_models = {}
    for company_id in feedback_df['company_id'].unique():
        company_data = feedback_df[feedback_df['company_id'] == company_id]
        bandit_models[company_id] = train_thompson_sampling(company_data)

    # 6. Save models to Redis
    save_models_to_redis({
        'global': global_model,
        'glmix': glmix,
        'bandits': bandit_models,
        'scaler': scaler,
        'version': get_current_timestamp()
    })

    print(f"Models trained and deployed. Version: {get_current_timestamp()}")

def train_thompson_sampling(company_data):
    """Train Thompson Sampling bandit per company"""

    # Cluster candidates into arms

```

```

candidates = company_data['candidate_id'].unique()
candidate_features = get_candidate_features(candidates)

kmeans = KMeans(n_clusters=10, random_state=42)
clusters = kmeans.fit_predict(candidate_features)

# Compute priors from feedback
arm_stats = {}
for arm_id in range(10):
    arm_candidates = candidates[clusters == arm_id]
    arm_feedback = company_data[company_data['candidate_id'].isin(
        arm_candidates)]

    n_success = (arm_feedback['event_type'] == 'hire').sum()
    n_failure = (arm_feedback['event_type'] == 'reject').sum()

    arm_stats[arm_id] = {
        'alpha': 1 + n_success,    # Beta prior alpha
        'beta': 1 + n_failure,    # Beta prior beta
        'cluster_center': kmeans.cluster_centers_[arm_id]
    }

return {
    'arm_stats': arm_stats,
    'kmeans': kmeans
}

```

## 4.2 Online Serving Pipeline

```

# serving.py

from fastapi import FastAPI, HTTPException
import redis
import pickle
import numpy as np

app = FastAPI()
redis_client = redis.Redis(host='localhost', port=6379, db=0)

@app.post("/api/v1/recommendations")
async def get_recommendations(request: RecommendationRequest):
    """
    Get personalized candidate recommendations for a job

    Request:
        job_id: UUID
        company_id: UUID
        recruiter_id: UUID
        top_k: int (default: 50)
        explain: bool (default: True)

    Response:
        candidates: List[CandidateRecommendation]
        metadata: RecommendationMetadata
    """

    # 1. Base matching (existing system)
    base_matches = await get_base_matches(request.job_id, top_k=500)

```

```
# 2. Load personalization models
models = load_models_from_redis()
company_model = models['glmix'].get_entity_model(request.company_id)

# 3. Check if cold start (use bandit)
company_history_count = get_company_hire_count(request.company_id)

if company_history_count < 20:
    # Cold start: Use multi-armed bandit
    reranked = apply_bandit_reranking(
        base_matches,
        request.company_id,
        models['bandits']
    )
else:
    # Warm start: Use GLMix
    reranked = apply_glmix_reranking(
        base_matches,
        request.job_id,
        company_model,
        models['scaler']
    )

# 4. Apply diversity filter (Maximal Marginal Relevance)
diverse_results = apply_mmr(reranked, lambda_param=0.7, top_k=request.
    top_k)

# 5. Generate explanations
if request.explain:
    explanations = generate_explanations(
        diverse_results,
        company_model,
        request.job_id
    )
else:
    explanations = None

# 6. Log recommendations for feedback loop
log_recommendation_event(
    job_id=request.job_id,
    company_id=request.company_id,
    candidates=[c.candidate_id for c in diverse_results]
)

return {
    'candidates': diverse_results,
    'explanations': explanations,
    'metadata': {
        'model_version': models['version'],
        'personalization_level': 'bandit' if company_history_count <
            20 else 'glmix',
        'diversity_applied': True
    }
}

def apply_glmix_reranking(candidates, job_id, company_model, scaler):
    """Apply GLMix personalization to rerank candidates"""
```

```

    job = get_job(job_id)
    features_list = []

    for candidate in candidates:
        features = extract_glmix_features(candidate, job, company_model)
        features_list.append(features)

    X = pd.DataFrame(features_list)
    X_scaled = scaler.transform(X)

    # Compute personalized scores
    scores = company_model.predict_proba(X_scaled)[: , 1] # Probability of hire

    # Combine with base scores
    final_scores = 0.6 * scores + 0.4 * np.array([c.base_score for c in candidates])

    # Rank
    ranked_indices = np.argsort(-final_scores)

    return [
        CandidateRecommendation(
            candidate_id=candidates[i].candidate_id,
            score=final_scores[i],
            base_score=candidates[i].base_score,
            personalization_score=scores[i]
        )
        for i in ranked_indices
    ]

def apply_mmr(candidates, lambda_param=0.7, top_k=50):
    """Apply Maximal Marginal Relevance for diversity"""

    selected = []
    remaining = candidates.copy()

    # Select first candidate (highest score)
    selected.append(remaining.pop(0))

    while len(selected) < top_k and remaining:
        # Compute MMR scores
        mmr_scores = []
        for candidate in remaining:
            relevance = candidate.score

            # Max similarity to already selected candidates
            max_similarity = max([
                compute_similarity(candidate, s)
                for s in selected
            ])

            mmr = lambda_param * relevance - (1 - lambda_param) * max_similarity
            mmr_scores.append(mmr)

        # Select candidate with highest MMR
        best_idx = np.argmax(mmr_scores)
        selected.append(remaining.pop(best_idx))

```

```
return selected
```

### 4.3 Feedback Collection

```
# feedback_collector.py

@app.post("/api/v1/feedback")
async def collect_feedback(feedback: FeedbackEvent):
    """
    Collect feedback events for model retraining

    Events:
    - view: Recruiter viewed candidate profile (implicit positive)
    - shortlist: Recruiter shortlisted candidate (explicit positive)
    - reject: Recruiter rejected candidate (explicit negative)
    - interview: Candidate invited to interview (strong positive)
    - hire: Candidate hired (strongest positive)
    """

    # Validate event
    validate_feedback_event(feedback)

    # Compute implicit score based on event type
    implicit_scores = {
        'view': 0.1,
        'shortlist': 0.5,
        'reject': 0.0,
        'interview': 0.8,
        'hire': 1.0
    }

    feedback.implicit_score = implicit_scores.get(feedback.event_type,
        0.0)

    # Store in database
    await store_feedback_event(feedback)

    # Update online bandit models (for real-time learning)
    if feedback.event_type in ['hire', 'reject']:
        await update_bandit_online(feedback)

    return {'status': 'success', 'event_id': feedback.event_id}

async def update_bandit_online(feedback: FeedbackEvent):
    """Update Thompson Sampling bandits in real-time"""

    # Load current bandit model
    bandit = load_bandit_from_redis(feedback.company_id)

    # Determine which arm this candidate belongs to
    candidate_features = get_candidate_features(feedback.candidate_id)
    arm_id = bandit['kmeans'].predict([candidate_features])[0]

    # Update arm statistics
    if feedback.event_type == 'hire':
        bandit['arm_stats'][arm_id]['alpha'] += 1
    elif feedback.event_type == 'reject':
```

```

        bandit['arm_stats'][arm_id]['beta'] += 1

# Save updated bandit
save_bandit_to_redis(feedback.company_id, bandit)

```

## 5 Explainability Module

### 5.1 Feature Attribution with SHAP

```

# explainability.py

import shap

def generate_explanations(recommendations, company_model, job_id):
    """Generate SHAP-based explanations for recommendations"""

    job = get_job(job_id)
    explanations = []

    # Create SHAP explainer
    explainer = shap.TreeExplainer(company_model)

    for rec in recommendations:
        candidate = get_candidate(rec.candidate_id)
        features = extract_glmix_features(candidate, job, company_model)

        # Compute SHAP values
        shap_values = explainer.shap_values(features)

        # Extract top contributing features
        feature_importance = dict(zip(features.keys(), shap_values))
        top_features = sorted(
            feature_importance.items(),
            key=lambda x: abs(x[1]),
            reverse=True
        )[:5]

        # Generate human-readable explanation
        explanation = generate_readable_explanation(
            candidate,
            job,
            top_features
        )

        explanations.append({
            'candidate_id': rec.candidate_id,
            'explanation': explanation,
            'top_features': top_features
        })

    return explanations

def generate_readable_explanation(candidate, job, top_features):
    """Convert SHAP features to human-readable text"""

    explanations = []

    for feature_name, importance in top_features:

```

```

    if feature_name == 'location_hist_match' and importance > 0:
        explanations.append(
            f"Strong match: {candidate.location} aligns with your past hires"
        )
    elif feature_name == 'skill_match_score' and importance > 0:
        explanations.append(
            f"Excellent skill alignment: {candidate.top_skills}"
        )
    elif feature_name == 'experience_years' and importance > 0:
        explanations.append(
            f"{candidate.total_experience} years experience matches your preference"
        )
    elif feature_name == 'college_hist_match' and importance > 0:
        explanations.append(
            f"Graduate of {candidate.education[0].college}, similar to past successful hires"
        )

    return " ".join(explanations)

```

## 6 Bias Mitigation & Fairness

### 6.1 Protected Attribute Filtering

```

# fairness.py

PROTECTED_ATTRIBUTES = [
    'gender', 'age', 'religion', 'caste', 'marital_status',
    'pregnancy_status', 'disability_status'
]

def filter_protected_attributes(features: dict) -> dict:
    """Remove protected attributes from feature set"""

    return {
        k: v for k, v in features.items()
        if k not in PROTECTED_ATTRIBUTES
    }

def validate_feature_set(features: dict):
    """Ensure no protected attributes in features"""

    forbidden = set(features.keys()) & set(PROTECTED_ATTRIBUTES)
    if forbidden:
        raise ValueError(f"Protected attributes found: {forbidden}")

```

### 6.2 Disparate Impact Monitoring

```

def compute_disparate_impact(recommendations, demographic_data):
    """
    Compute disparate impact ratio for protected groups

    Rule: Ratio should be >= 0.8 (80% rule)
    """

```

```

groups = ['male', 'female', 'non-binary']
selection_rates = {}

for group in groups:
    group_candidates = [
        c for c in recommendations
        if demographic_data[c.candidate_id]['gender'] == group
    ]

    # Selection rate = % of top-50 from this group
    selection_rates[group] = len(group_candidates) / len(
        recommendations)

    # Compute disparate impact
    max_rate = max(selection_rates.values())
    disparate_impacts = {
        group: rate / max_rate
        for group, rate in selection_rates.items()
    }

    # Flag violations
    violations = {
        group: impact
        for group, impact in disparate_impacts.items()
        if impact < 0.8
    }

    if violations:
        alert_compliance_team(violations)

return disparate_impacts

```

### 6.3 Diversity Injection

```

def ensure_diversity(recommendations, min_diversity=0.2):
    """
    Ensure at least min_diversity of recommendations are "exploratory"
    (i.e., from different clusters than historical hires)
    """

    num_explore = int(len(recommendations) * min_diversity)

    # Identify exploration candidates (low historical similarity)
    exploration_candidates = [
        c for c in recommendations
        if c.historical_similarity < 0.5
    ]

    # If insufficient diversity, force-inject exploration candidates
    if len(exploration_candidates) < num_explore:
        # Fetch additional diverse candidates
        additional = fetch_diverse_candidates(
            num_needed=num_explore - len(exploration_candidates),
            exclude_ids=[c.candidate_id for c in recommendations]
        )

        # Replace bottom-ranked recommendations with exploration

```

```
        recommendations = (  
            recommendations[:-len(additional)] +  
            additional  
        )  
  
    return recommendations
```

## 7 Performance Optimization

### 7.1 Caching Strategy

```
# caching.py  
  
import redis  
from functools import wraps  
  
redis_client = redis.Redis(host='localhost', port=6379, db=1)  
  
def cache_recommendations(ttl=3600): # 1 hour TTL  
    """Cache recommendation results"""  
  
    def decorator(func):  
        @wraps(func)  
        async def wrapper(job_id, company_id, *args, **kwargs):  
            # Generate cache key  
            cache_key = f"recs:{job_id}:{company_id}"  
  
            # Check cache  
            cached = redis_client.get(cache_key)  
            if cached:  
                return pickle.loads(cached)  
  
            # Compute fresh recommendations  
            result = await func(job_id, company_id, *args, **kwargs)  
  
            # Cache result  
            redis_client.setex(  
                cache_key,  
                ttl,  
                pickle.dumps(result)  
            )  
  
            return result  
  
        return wrapper  
    return decorator  
  
@cache_recommendations(ttl=1800) # 30 min cache  
async def get_recommendations(job_id, company_id, top_k=50):  
    # ... recommendation logic ...  
    pass
```

### 7.2 Async Processing

```
# async_processing.py
```

```
import asyncio
from concurrent.futures import ThreadPoolExecutor

executor = ThreadPoolExecutor(max_workers=10)

async def get_recommendations_async(job_id, company_id):
    """Parallelize independent operations"""

    # Fetch data in parallel
    base_matches_task = asyncio.create_task(get_base_matches(job_id))
    models_task = asyncio.create_task(load_models_from_redis())
    history_task = asyncio.create_task(get_company_history(company_id))

    base_matches, models, history = await asyncio.gather(
        base_matches_task,
        models_task,
        history_task
    )

    # Reranking (CPU-intensive, offload to thread pool)
    reranked = await asyncio.get_event_loop().run_in_executor(
        executor,
        apply_glmix_reranking,
        base_matches,
        job_id,
        models,
        history
    )

    return reranked
```

## 8 Monitoring & Observability

### 8.1 Key Metrics to Track

```
# monitoring.py

from prometheus_client import Counter, Histogram, Gauge

# Latency metrics
recommendation_latency = Histogram(
    'recommendation_latency_seconds',
    'Time to generate recommendations',
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0]
)

# Throughput metrics
recommendations_total = Counter(
    'recommendations_total',
    'Total recommendations served',
    ['company_id', 'model_type']
)

# Quality metrics
hire_rate = Gauge(
    'hire_rate',
    'Percentage of recommended candidates hired',
    ['company_id', 'time_window']
)
```

```

)

# Model health metrics
model_drift = Gauge(
    'model_drift_score',
    'Model_drift_detection_score',
    ['company_id', 'model_version']
)

# Fairness metrics
disparate_impact = Gauge(
    'disparate_impact_ratio',
    'Disparate_impact_ratio_for_protected_groups',
    ['company_id', 'group']
)

```

## 8.2 Alerting Rules

- **Latency:** Alert if p99 latency  $\geq$  200ms
- **Error Rate:** Alert if error rate  $\geq$  1%
- **Model Drift:** Alert if drift score  $\geq$  0.3
- **Fairness:** Alert if disparate impact  $\geq$  0.8 for any group
- **Hire Rate:** Alert if hire rate drops by  $\geq$  20% week-over-week

## 9 Deployment Strategy

### 9.1 Rollout Plan

1. **Week 1-2:** Deploy to staging, integration testing
2. **Week 3:** A/B test with 5% of traffic (shadow mode)
3. **Week 4:** Analyze A/B results, fix issues
4. **Week 5-6:** Gradual rollout to 25%, 50%, 100% of traffic
5. **Week 7+:** Monitor and iterate

### 9.2 A/B Testing Framework

```

# ab_testing.py

import hashlib

def assign_to_variant(user_id, experiment_name):
    """Deterministic assignment to A/B test variant"""

    hash_input = f"{user_id}:{experiment_name}"
    hash_value = int(hashlib.md5(hash_input.encode()).hexdigest(), 16)

    # 50/50 split
    return 'treatment' if hash_value % 2 == 0 else 'control'

@app.post("/api/v1/recommendations")

```

```
async def get_recommendations(request):
    variant = assign_to_variant(request.company_id, 'personalization_v1')

    if variant == 'treatment':
        # Use personalized recommendations
        results = await get_personalized_recommendations(request)
    else:
        # Use base matching only
        results = await get_base_recommendations(request)

    # Log for experiment analysis
    log_experiment_event(
        company_id=request.company_id,
        variant=variant,
        results=results
    )

    return results
```

## 10 Technology Stack

### 10.1 Core Components

| Component         | Technology                                  |
|-------------------|---------------------------------------------|
| API Server        | FastAPI (Python 3.11)                       |
| Vector Database   | FAISS (Facebook AI)                         |
| Model Store       | Redis (with persistence)                    |
| Feedback Database | PostgreSQL 15                               |
| Model Training    | Scikit-learn, XGBoost, GLMix                |
| Bandit Algorithms | Custom implementation + MABWiser            |
| Explainability    | SHAP                                        |
| Monitoring        | Prometheus + Grafana                        |
| Logging           | ELK Stack (Elasticsearch, Logstash, Kibana) |
| Orchestration     | Kubernetes                                  |
| CI/CD             | GitHub Actions                              |

### 10.2 Infrastructure Requirements

**Production Environment:**

- **API Servers:** 4 pods × 4 CPU, 8 GB RAM
- **Redis:** 1 instance × 8 CPU, 16 GB RAM, 100 GB SSD
- **PostgreSQL:** 1 instance × 8 CPU, 32 GB RAM, 500 GB SSD
- **FAISS:** Shared with existing matching system
- **Model Training:** 1 GPU instance (T4) for weekly retraining

**Estimated Costs:**

- **Compute:** \$3,000/month (AWS/GCP)
- **Storage:** \$500/month
- **Monitoring/Logging:** \$500/month
- **Total:** \$4,000-\$5,000/month

## 11 Testing Strategy

### 11.1 Unit Tests

```
# tests/test_glmix.py

import pytest
from models.glmix import GLMixModel

def test_glmix_training():
    """Test GLMix model training"""

    # Mock data
    X = np.random.randn(1000, 20)
    y = np.random.randint(0, 2, 1000)
    entity_ids = np.random.randint(0, 10, (1000, 2))

    model = GLMixModel()
    model.fit(X, y, entity_ids)

    # Test predictions
    preds = model.predict_proba(X[:10])

    assert preds.shape == (10, 2)
    assert np.all((preds >= 0) & (preds <= 1))
    assert np.allclose(preds.sum(axis=1), 1.0)

def test_thompson_sampling():
    """Test Thompson Sampling bandit"""

    bandit = ThompsonSamplingBandit(n_arms=5)

    for _ in range(100):
        arm = bandit.select_arm()
        reward = np.random.binomial(1, 0.5)
        bandit.update(arm, reward)

    # After 100 trials, should have non-uniform arm selection
    arm_counts = bandit.get_arm_counts()
    assert max(arm_counts) > min(arm_counts)
```

### 11.2 Integration Tests

```
# tests/test_api.py

from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_recommendations_endpoint():
    """Test recommendations API endpoint"""

    response = client.post("/api/v1/recommendations", json={
        "job_id": "test-job-123",
        "company_id": "test-company-456",
        "top_k": 50
    })
```

```
assert response.status_code == 200
data = response.json()

assert 'candidates' in data
assert len(data['candidates']) <= 50
assert all('candidate_id' in c for c in data['candidates'])
assert all('score' in c for c in data['candidates'])
```

### 11.3 Load Testing

```
# load_test.py using Locust

from locust import HttpUser, task, between

class RecommendationUser(HttpUser):
    wait_time = between(1, 3)

    @task
    def get_recommendations(self):
        self.client.post("/api/v1/recommendations", json={
            "job_id": "test-job-123",
            "company_id": "test-company-456",
            "top_k": 50
        })

# Run: locust -f load_test.py --host=http://localhost:8000
# Target: 1000 requests/second with p99 latency < 200ms
```

## 12 Conclusion

This technical architecture provides a comprehensive, production-ready implementation of a personalized recommendation layer for CV-job matching. Key highlights:

- **Non-disruptive:** Builds on existing infrastructure
- **Scalable:** Handles millions of recommendations per day
- **Fair:** Built-in bias monitoring and mitigation
- **Explainable:** SHAP-based feature attribution
- **Low-latency:**  $\leq 50$ ms additional overhead with caching
- **Battle-tested:** Based on LinkedIn's proven GLMix + Bandit approach

*For technical questions, contact the Engineering Team*